

Lab Report No10
Discrete Fourier Transform

Digital Signal Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

Submitted to:

Engr. Hareem Khan

Dated:

17th May, 2024

Department of Computer Engineering,
HITEC University, Taxila

Lab Example No 1:

Solution:

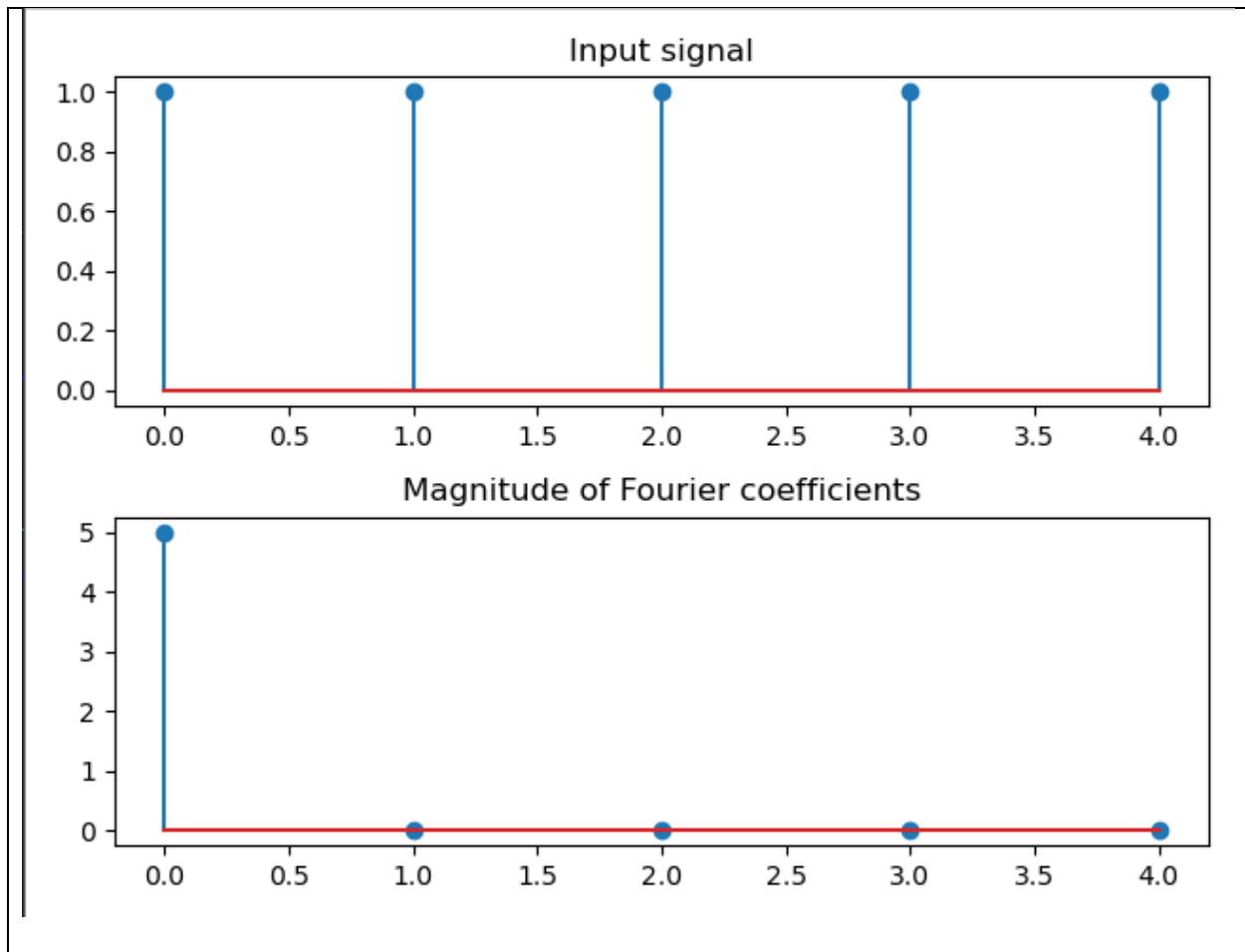
Brief description (3-5 lines)

The Fourier transform is a mathematical tool used for analyzing an arbitrary aperiodic signal by decomposing it into a weighted sum of much simpler sinusoidal component functions. The weights, or coefficients, are a one-to-one mapping of the original function. Fourier transform serve many useful purposes, as manipulation and conceptualization of the coefficients are often easier than with the original function.

The code

```
Fouriertransform.py > ...  
  Click here to ask Blackbox to help you code faster  
1  import numpy as np  
2  import matplotlib.pyplot as plt  
3  y=np.ones(5)  
4  Y=np.fft.fft(y)  
5  plt.subplot(2,1,1)  
6  plt.stem(y)  
7  plt.title('Input signal')  
8  plt.subplot(2,1,2)  
9  plt.stem(np.abs(Y))  
10 plt.title('Magnitude of Fourier coefficients')  
11 plt.tight_layout()  
12 plt.show()
```

The results (Screenshot)



Lab Example No 2:

Solution:

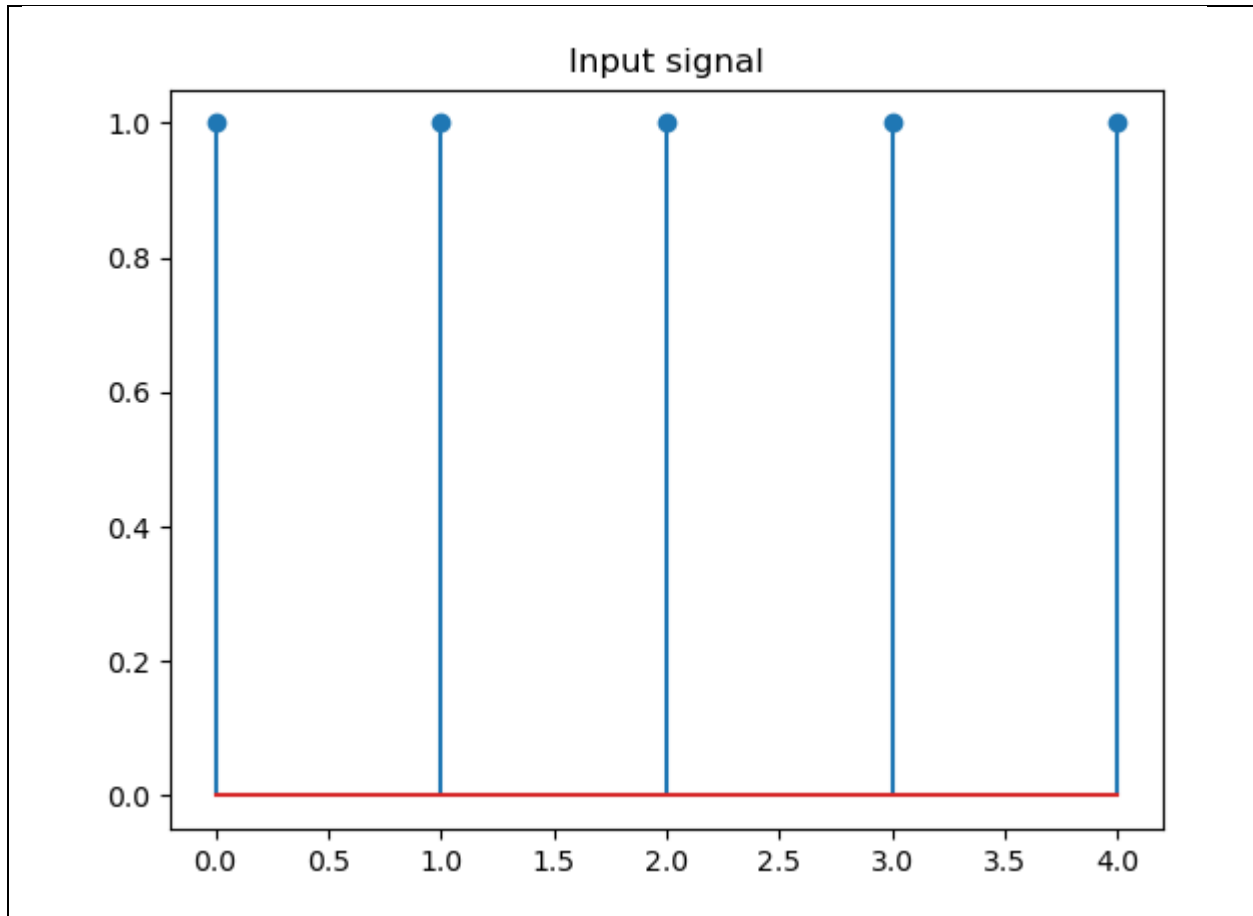
Brief description (3-5 lines)
Decomposition: (a) Write a python code to decompose the signal into its complex exponentials. b. Verify your results with built-in fft command. <code>stem(abs(fft(y,5)));</code>
The code

DECOMPOSITION.PY > ...

Click here to ask Blackbox to help you code faster

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 y=np.ones(5)
4 plt.stem(y)
5 plt.title('Input signal')
6 plt.show()
7 N=len(y)
8 n=np.arange(N)
9 k=n.reshape((N,1))
10 exp_term=np.exp(-2j*np.pi*k*n/N)
11 decomp=np.dot(exp_term,y)
12 plt.stem(np.real(decomp))
13 plt.title('Output signal from decomposition')
14 plt.show()
15 Y_fft=np.fft.fft(y)
16 plt.stem(np.abs(Y_fft))
17 plt.title('Magnitude of fourier coefficents using built-in FFT')
18 plt.show()
19 diff=np.abs(Y_fft)-np.abs(decomp)
20 plt.stem(diff)
21 plt.title('Difference between the output siganl and the FFT results')
22 plt.show()
```

The results (Screenshot)



Lab Example No 3(a):

Solution:

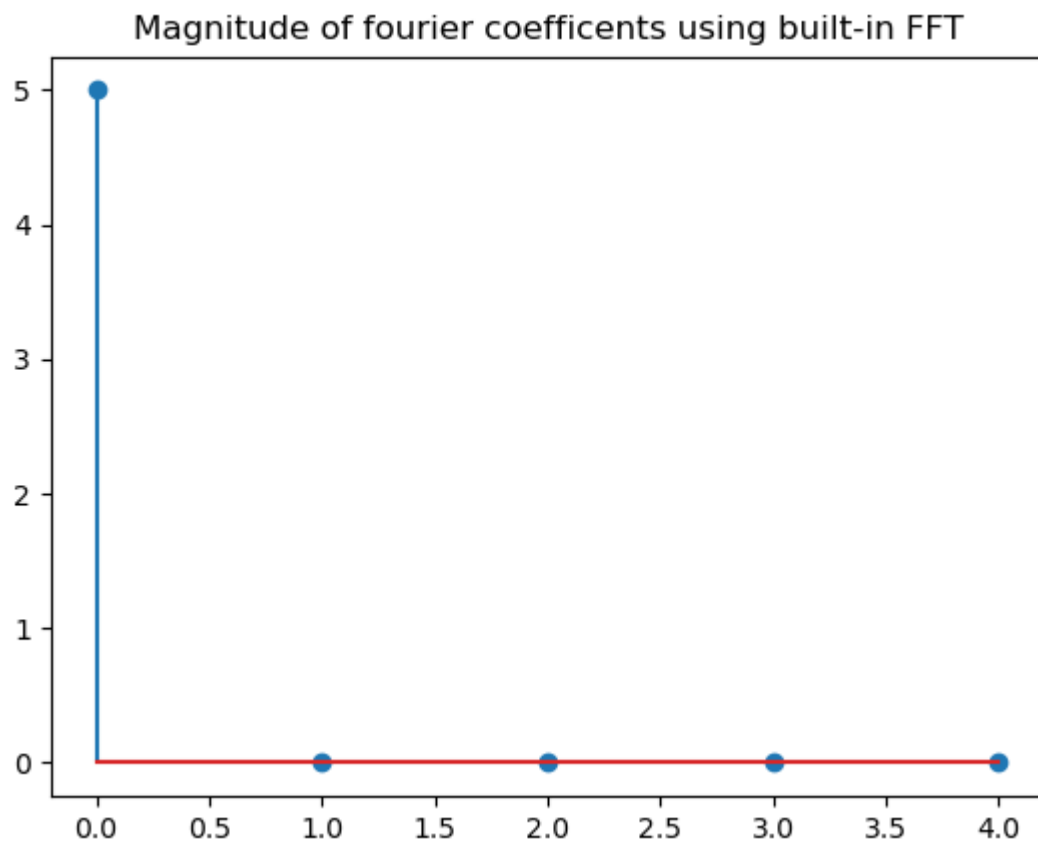
Brief description (3-5 lines)
Reconstruction: a. Write a python code to add Fourier series coefficients calculated above. b. Compare the reconstructed signal with the original signal.
The code

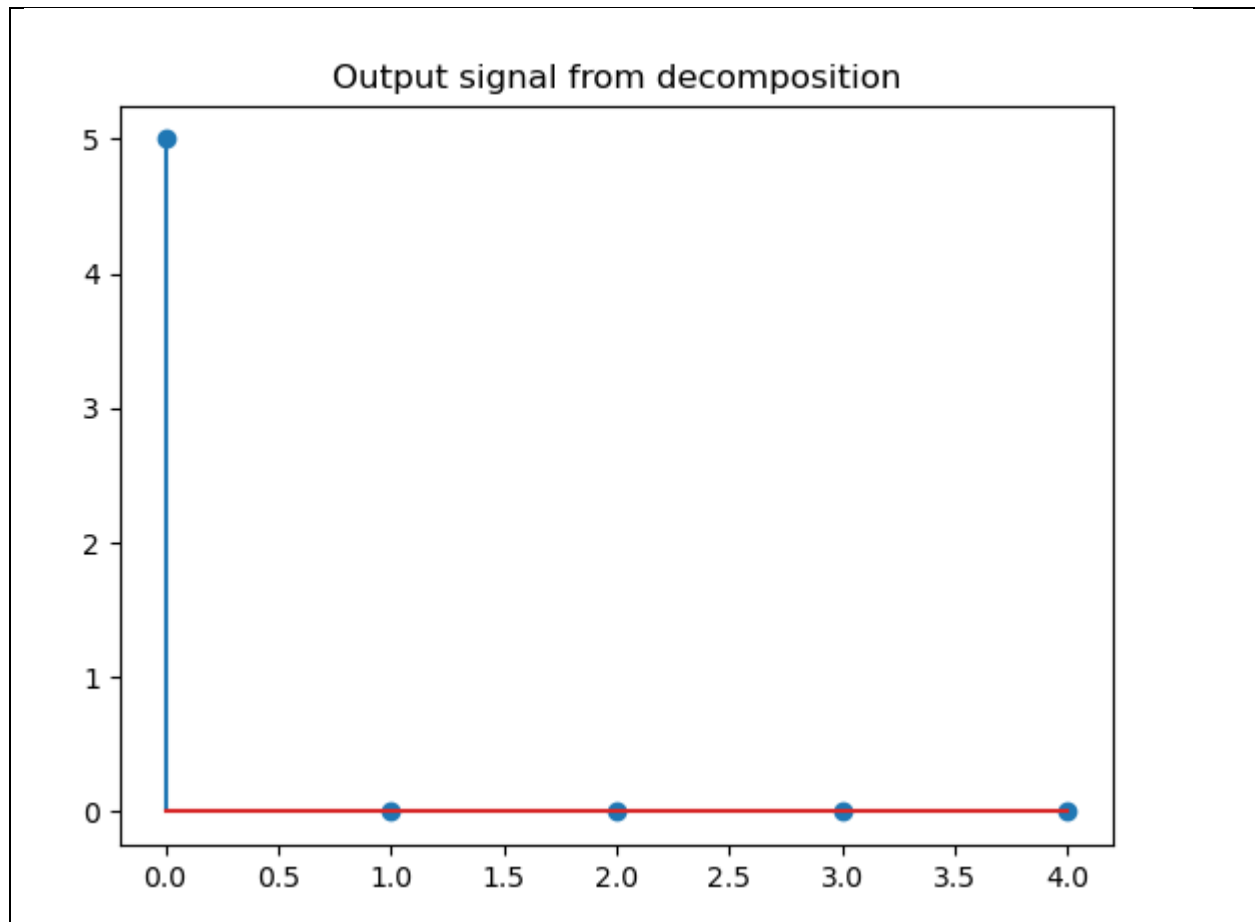
```

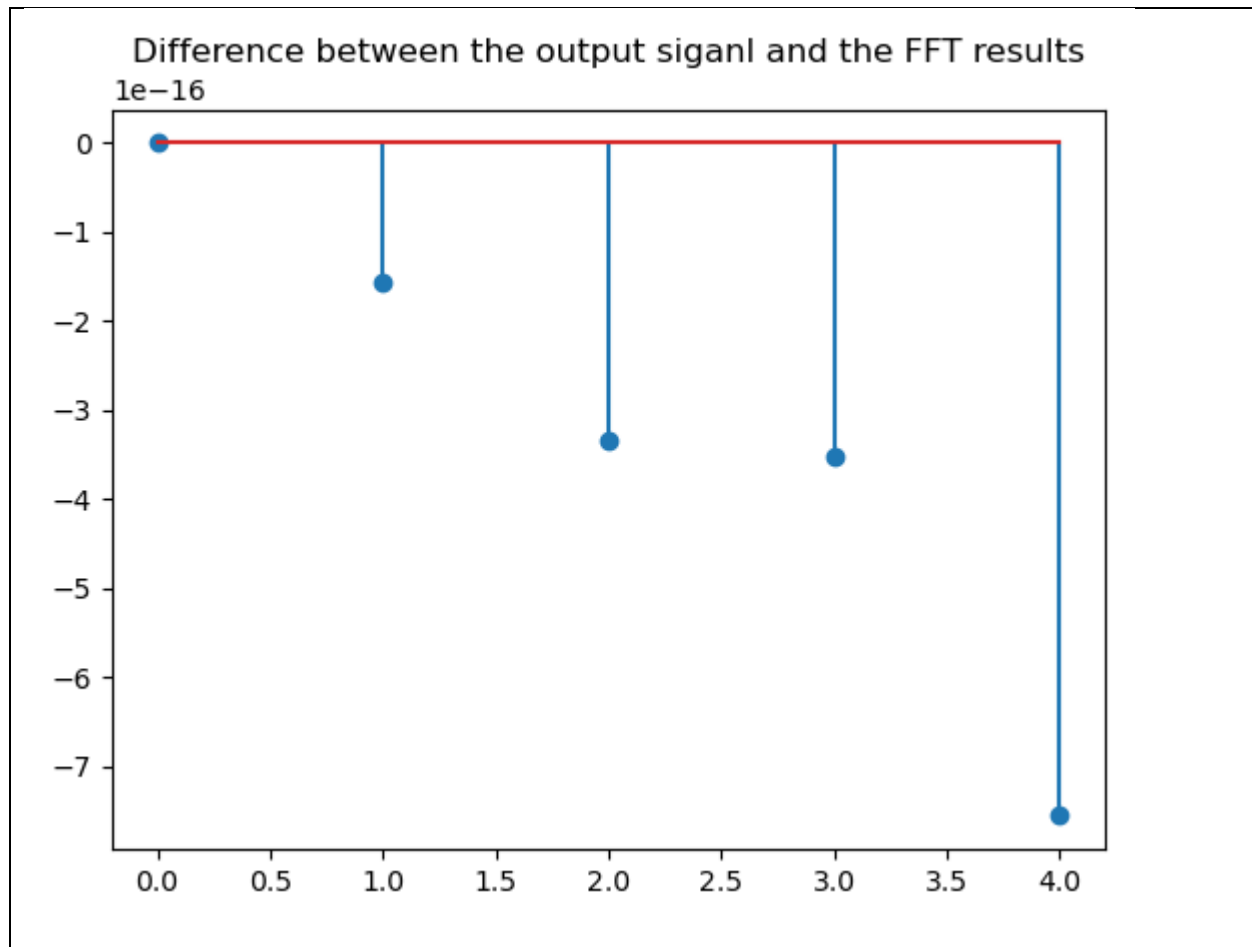
Reconstruction.py > ...
  Click here to ask Blackbox to help you code faster
1  import numpy as np
2  import matplotlib.pyplot as plt
3  y=np.ones(5)
4  plt.stem(y)
5  plt.title('Input signal')
6  plt.show()
7  N=len(y)
8  n=np.arange(N)
9  k=n.reshape((N,1))
10 exp_term=np.exp(-2j*np.pi*k*n/N)
11 coeff=np.dot(exp_term,y)
12 recon=np.dot(exp_term.T,coeff)
13 plt.stem(np.real(recon))
14 plt.title('Output signal from Fourier series coefficients')
15 plt.show()
16 diff=np.abs(y)-np.abs(recon)
17 plt.stem(diff)
18 plt.title('Difference between the original and reconstucted signals')
19 plt.show()

```

The results (Screenshot)







Lab Example No 3(b):

Solution:

Brief description (3-5 lines)

Reconstruction:

b. Compare the reconstructed signal with the original signal.

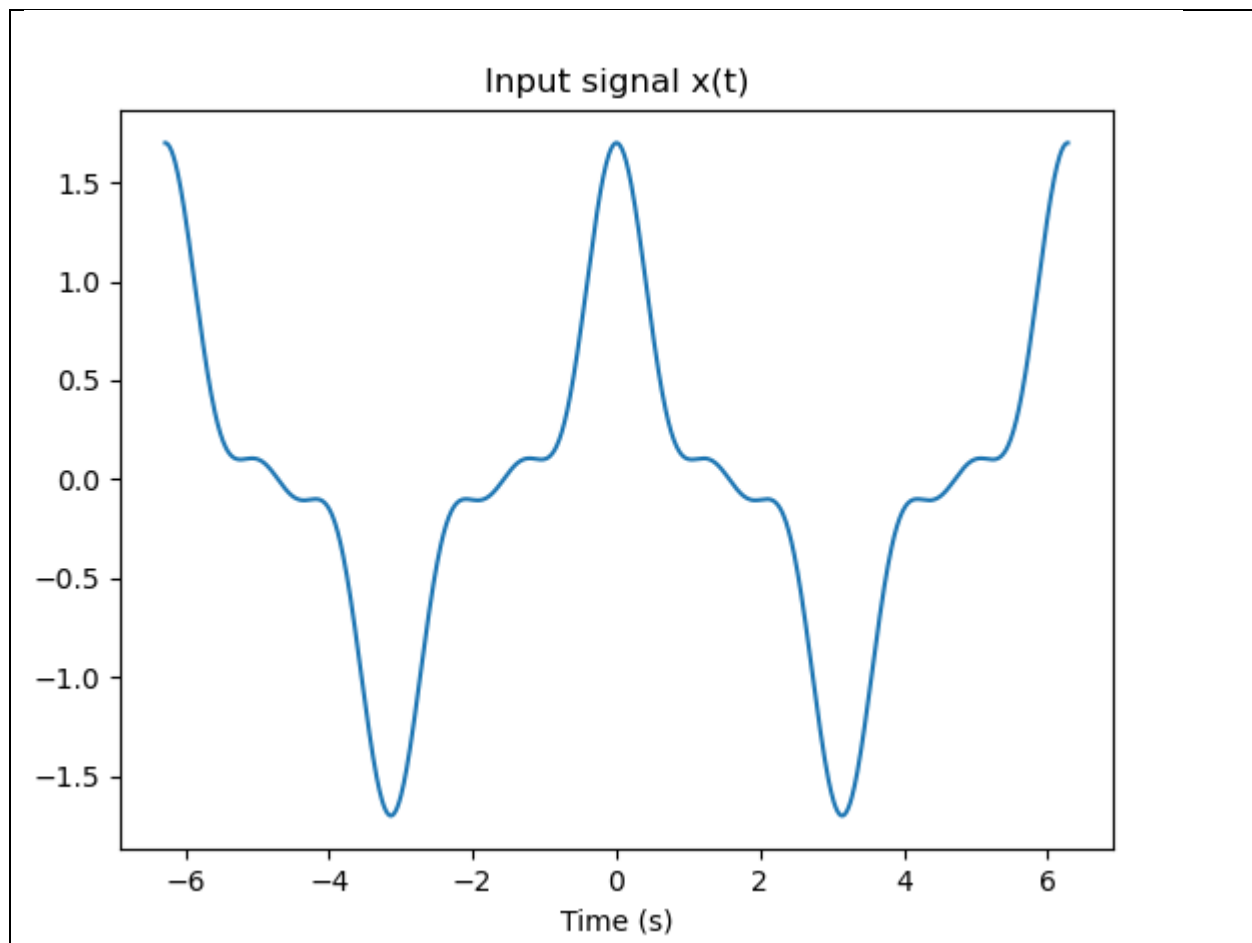
The code

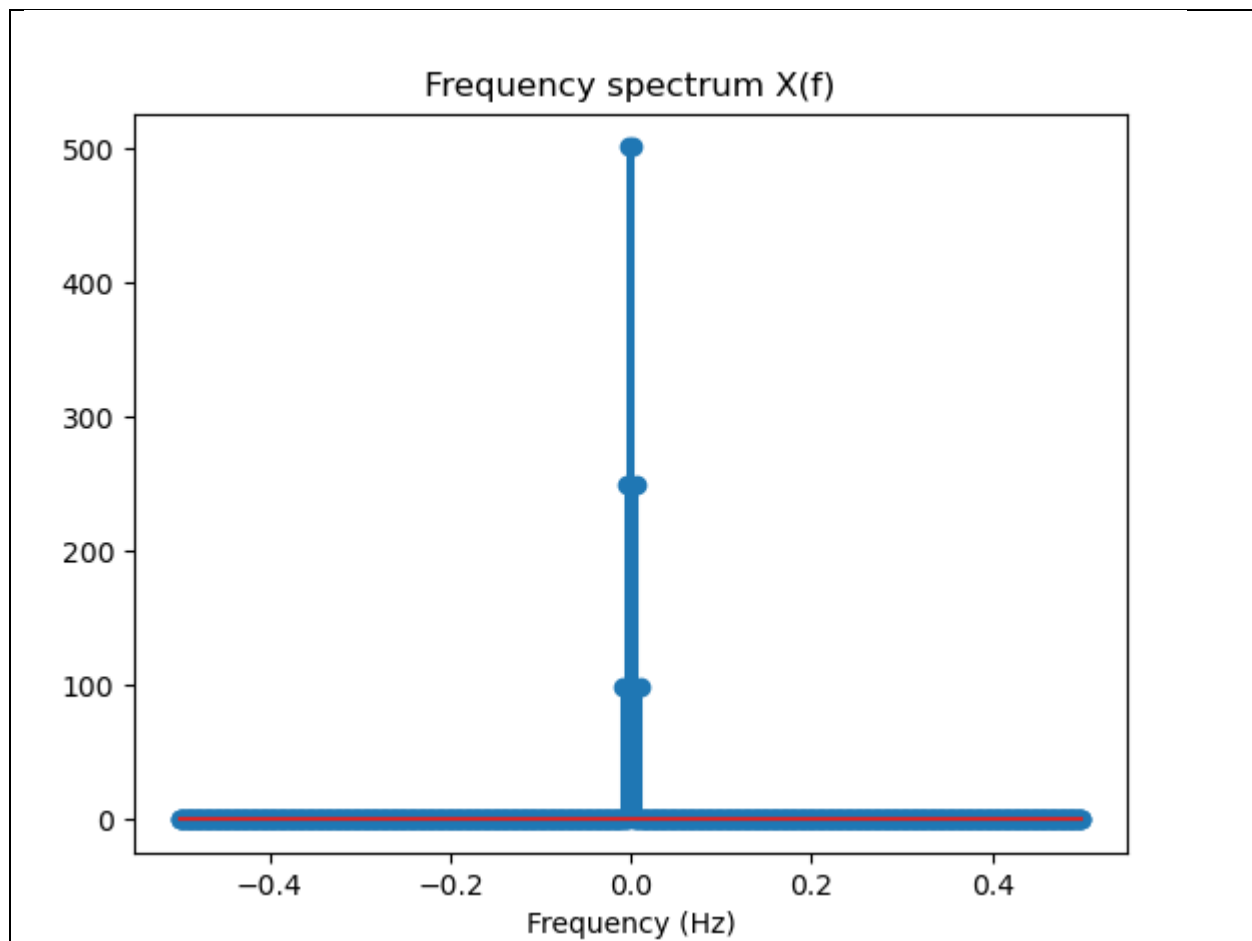
Reconstruction1.py > ...

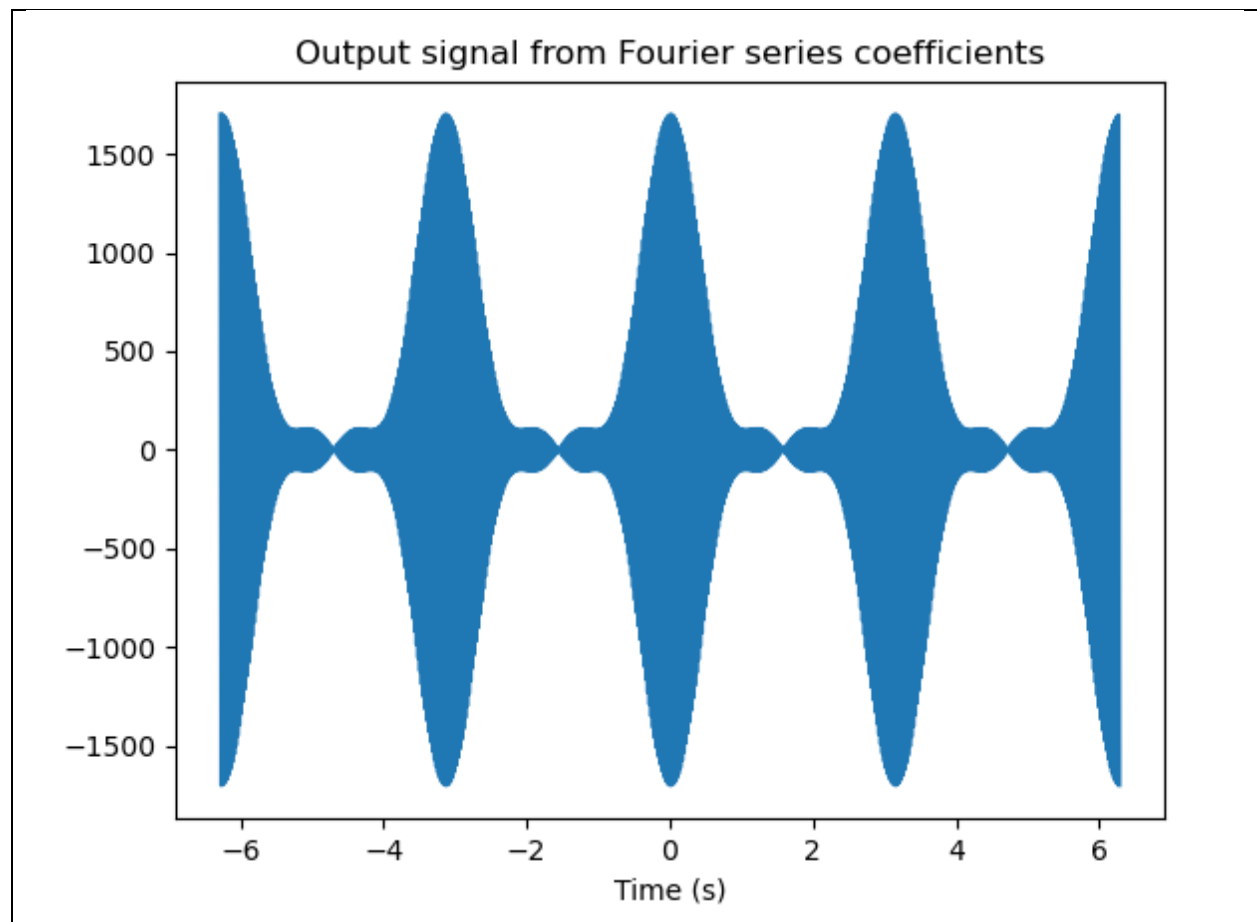
Click here to ask Blackbox to help you code faster

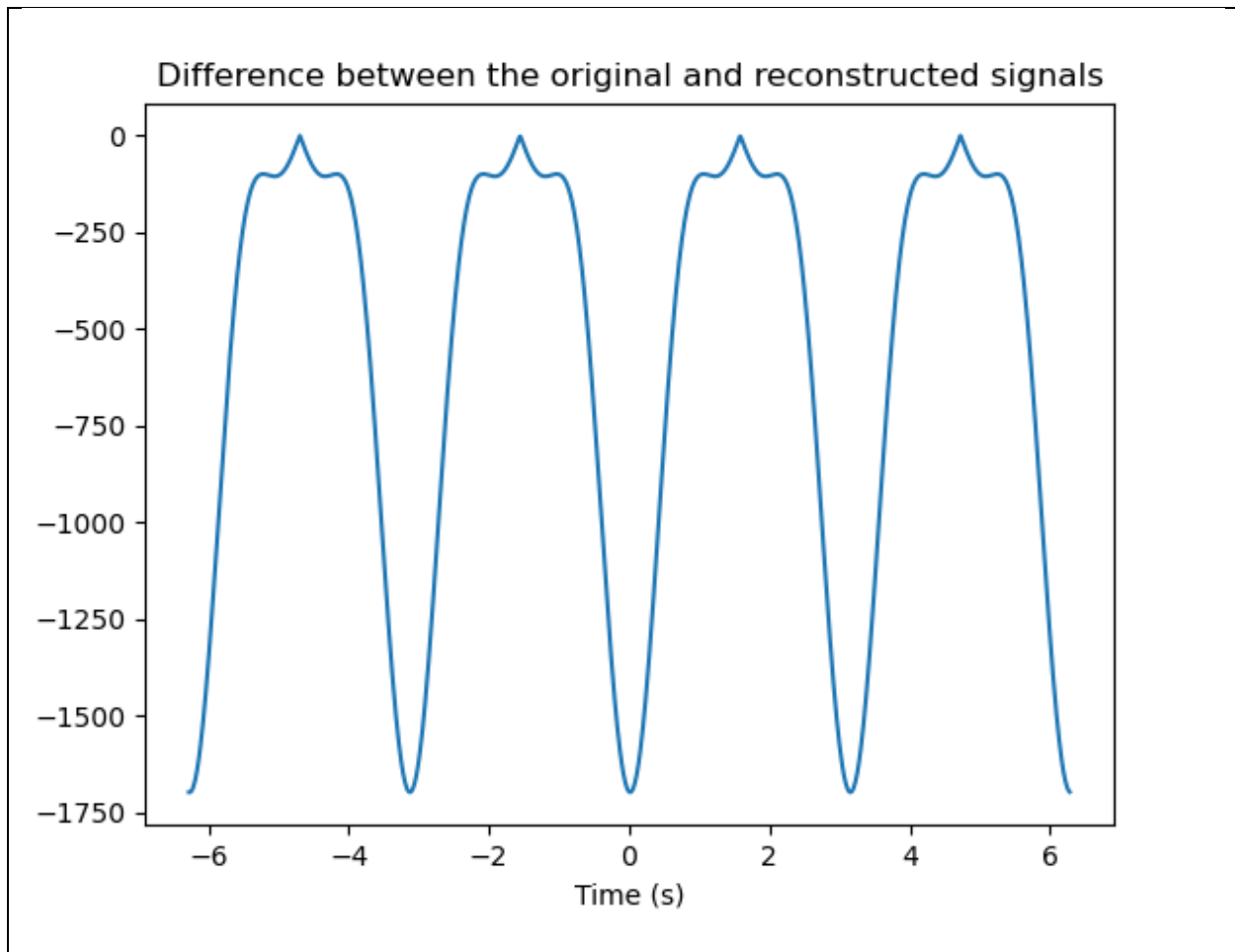
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 t = np.linspace(-2*np.pi, 2*np.pi, 1000)
4 x = np.cos(t) + 0.5*np.cos(3*t) + 0.2*np.cos(5*t)
5 plt.plot(t, x)
6 plt.title('Input signal x(t)')
7 plt.xlabel('Time (s)')
8 plt.show()
9 N = len(x)
10 n = np.arange(N)
11 k = n.reshape((N, 1))
12 exp_term = np.exp(-2j * np.pi * k * n / N)
13 coeff = np.dot(exp_term, x)
14 freq = np.fft.fftfreq(N)
15 idx = np.argsort(freq)
16 freq = freq[idx]
17 coeff = coeff[idx]
18 plt.stem(freq, np.abs(coeff))
19 plt.title('Frequency spectrum X(f)')
20 plt.xlabel('Frequency (Hz)')
21 plt.show()
22 recon = np.dot(exp_term.T, coeff)
23 plt.plot(t, recon)
24 plt.title('Output signal from Fourier series coefficients')
25 plt.xlabel('Time (s)')
26 plt.show()
27 diff = np.abs(x) - np.abs(recon)
28 plt.plot(t, diff)
29 plt.title('Difference between the original and reconstructed signals')
30 plt.xlabel('Time (s)')
31 plt.show()
```

The results (Screenshot)









Lab Task No 1:

Solution:

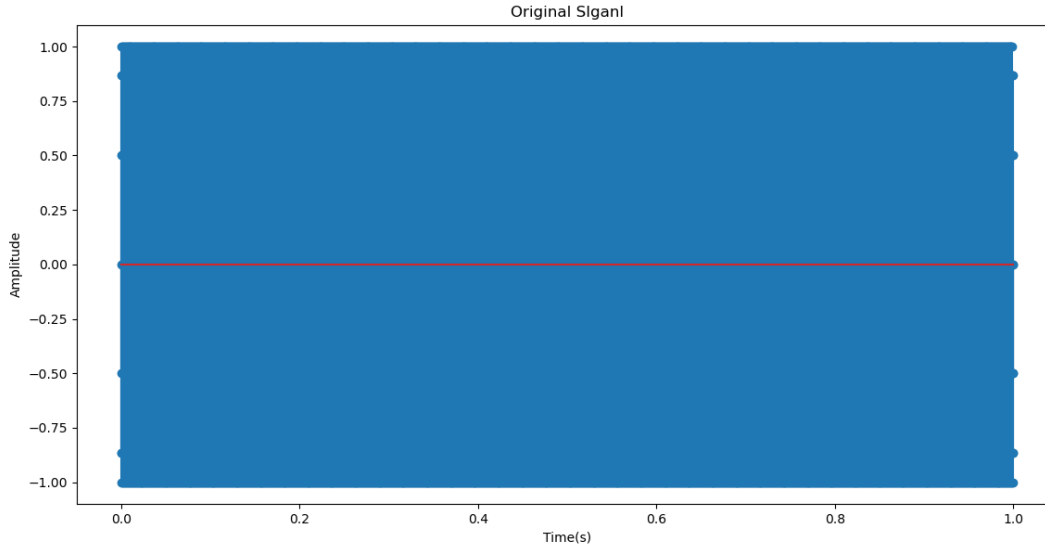
Brief description (3-5 lines)
a. Generate a sinusoid of 0.5 KHz with sampling frequency of 6 KHz.
b. Plot the original signal using stem.
c. Compute its complex exponentials.
d. Compare the magnitudes of the co-efficients of complex exponentials with those of built in command.
e. Compare the angle of the complex exponentials with those of built in command.
The code

FOURIERSERIESTRANSFORM.PY > ...

Click here to ask Blackbox to help you code faster

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 frequency=1000
4 sampling_freq=12000
5 duration=1
6 t=np.arange(0,duration,1/sampling_freq)
7 signal=np.cos(2*np.pi*frequency*t)
8 plt.stem(t,signal)
9 plt.xlabel('Time(s)')
10 plt.ylabel('Amplitude')
11 plt.title('Original Siganl')
12 plt.show()
13 exponential=np.exp(1j*2*np.pi*frequency*t)
14 magnitudes=np.abs(exponential)
15 angles=np.angle(exponential)
16 manual_magnitudes=np.sqrt(np.real(exponential)**2+np.imag(exponential)**2)
17 manual_angles=np.arctan2(np.imag(exponential),np.real(exponential))
18 print('Magnitudes(Built-in):',magnitudes[:10])
19 print('Magnitudes(Manual):',manual_magnitudes[:10])
20 print('Angles(Built-in):',angles[:10])
21 print('Angles(Manual):',manual_angles[:10])
```

The results (Screenshot)



Lab Task No 2:

Solution:

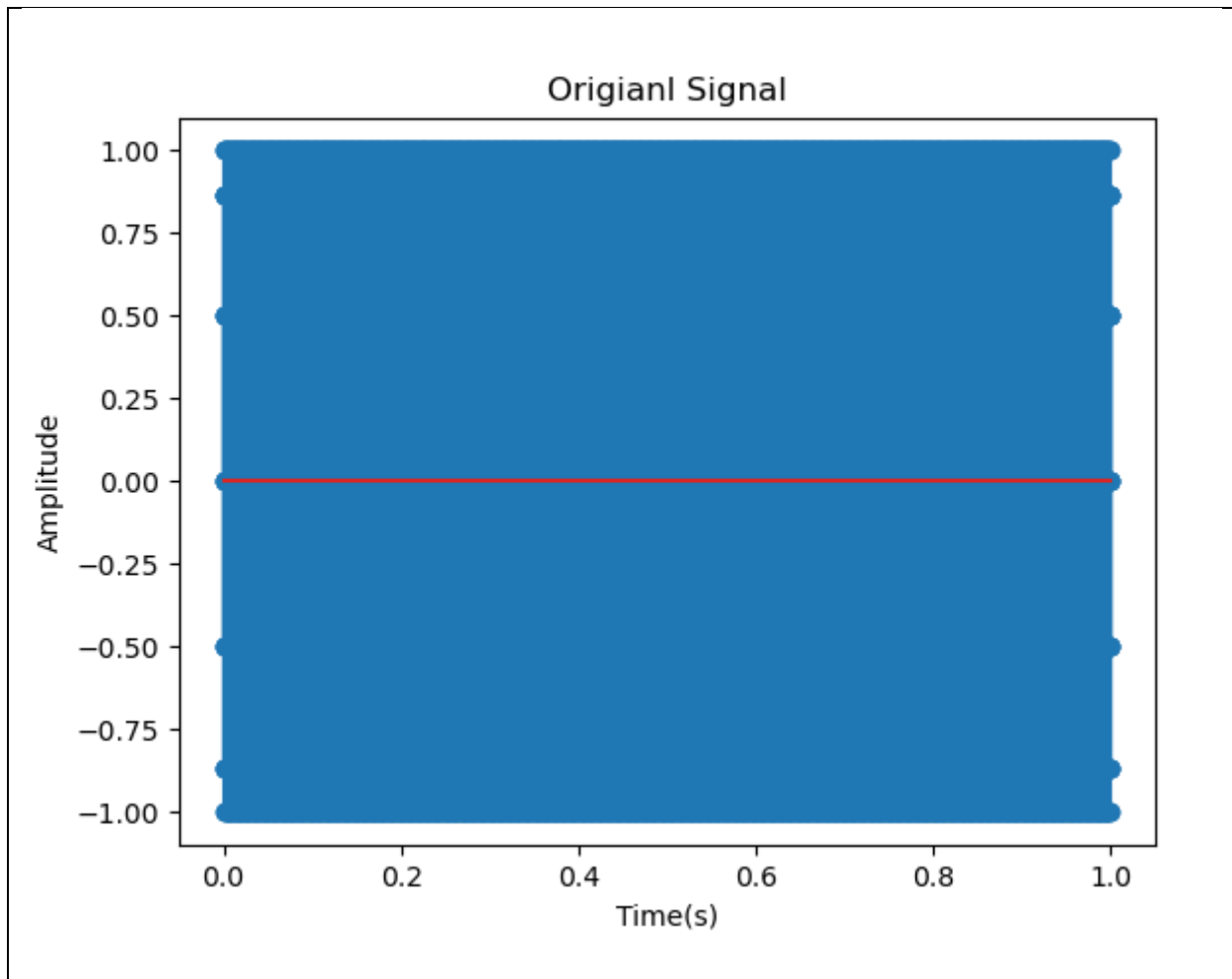
Brief description (3-5 lines)

- Generate a cosine signal of 1 KHz with sampling frequency of 12 KHz.
- Plot the original signal using stem.
- Compute its complex exponentials.
- Compare the magnitudes of the coefficients of complex exponentials with those of built in command.
- Compare the angle of the complex exponentials with those of built in command.

The code

```
MAGNITUDES.PY > ...  
  Click here to ask Blackbox to help you code faster  
1  import numpy as np  
2  import matplotlib.pyplot as plt  
3  frequency=1000  
4  sampling_freq=12000  
5  duration=1  
6  t=np.arange(0,duration,1/sampling_freq)  
7  signal=np.cos(2*np.pi*frequency*t)  
8  plt.stem(t,signal)  
9  plt.xlabel('Time(s)')  
10 plt.ylabel('Amplitude')  
11 plt.title('Original Signal')  
12 plt.show()  
13 exponentials=np.exp(1j*2*np.pi*frequency*t)  
14 magnitudes=np.abs(exponentials)  
15 angles=np.angle(exponentials)  
16 manual_magnitudes=np.sqrt(np.real(exponentials)**2+np.imag(exponentials)**2)  
17 manual_angles=np.arctan2(np.imag(exponentials),np.real(exponentials))  
18 print('Magnitudes(Built-in):',magnitudes[:10])  
19 print('Magnitudes(Manual):',manual_magnitudes[:10])  
20 print('Angles(Built-in):',angles[:10])  
21 print('Angles(Manual):',manual_angles[:10])
```

The results (Screenshot)



Lab Task No 3:

Solution:

Brief description (3-5 lines)
(a) Generate a sinusoid of 2.5 KHz with sampling frequency of 20 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.
(b) Generate a square wave of frequency 1 KHz with sampling frequency of 12 KHz. Plot the original signal using stem. Compute its

complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

(c) Generate a sawtooth wave of frequency 3 KHz with sampling frequency of 15 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

(d) Generate a triangle wave of frequency 1.2 KHz with sampling frequency of 18 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

e) Generate a random signal with Gaussian distribution and sampling frequency of 6 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

The code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
def plot_signal(t, x, title):
    plt.figure()
    plt.stem(t[:100], x[:100], use_line_collection=True)
    plt.title(title)
    plt.xlabel('Time [s]')
    plt.ylabel('Amplitude')
    plt.show()
def plot_spectrum(frequencies, X, title):
    magnitudes = np.abs(X)
    angles = np.angle(X)
    plt.figure()
    plt.stem(frequencies[:len(frequencies)//2], magnitudes[:len(frequencies)//2],
    use_line_collection=True)
    plt.title(f'Magnitude Spectrum of {title}')
    plt.xlabel('Frequency [Hz]')
    plt.ylabel('Magnitude')
    plt.show()
    plt.figure()
    plt.stem(frequencies[:len(frequencies)//2], angles[:len(frequencies)//2],
    use_line_collection=True)
```

```

plt.title(f'Phase Spectrum of {title}')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Angle [rad]')
plt.show()
# Part (a): Generate a Sinusoid
fs = 20000
f = 2500
t = np.arange(0, 1, 1/fs)
x = np.sin(2 * np.pi * f * t)
plot_signal(t, x, 'Sinusoid (2.5 KHz)')
X = np.fft.fft(x)
frequencies = np.fft.fftfreq(len(x), 1/fs)
plot_spectrum(frequencies, X, 'Sinusoid')
# Part (b): Generate a Square Wave
fs = 12000
f = 1000
t = np.arange(0, 1, 1/fs)
x = signal.square(2 * np.pi * f * t)
plot_signal(t, x, 'Square Wave (1 KHz)')
X = np.fft.fft(x)
frequencies = np.fft.fftfreq(len(x), 1/fs)
plot_spectrum(frequencies, X, 'Square Wave')
# Part (c): Generate a Sawtooth Wave
fs = 15000
f = 3000
t = np.arange(0, 1, 1/fs)
x = signal.sawtooth(2 * np.pi * f * t)
plot_signal(t, x, 'Sawtooth Wave (3 KHz)')
X = np.fft.fft(x)
frequencies = np.fft.fftfreq(len(x), 1/fs)
plot_spectrum(frequencies, X, 'Sawtooth Wave')
# Part (d): Generate a Triangle Wave
fs = 18000
f = 1200
t = np.arange(0, 1, 1/fs)
x = signal.sawtooth(2 * np.pi * f * t, 0.5)
plot_signal(t, x, 'Triangle Wave (1.2 KHz)')
X = np.fft.fft(x)
frequencies = np.fft.fftfreq(len(x), 1/fs)
plot_spectrum(frequencies, X, 'Triangle Wave')

```

```
# Part (e): Generate a Random Signal with Gaussian Distribution
```

```
fs = 6000
```

```
t = np.arange(0, 1, 1/fs)
```

```
x = np.random.normal(0, 1, len(t))
```

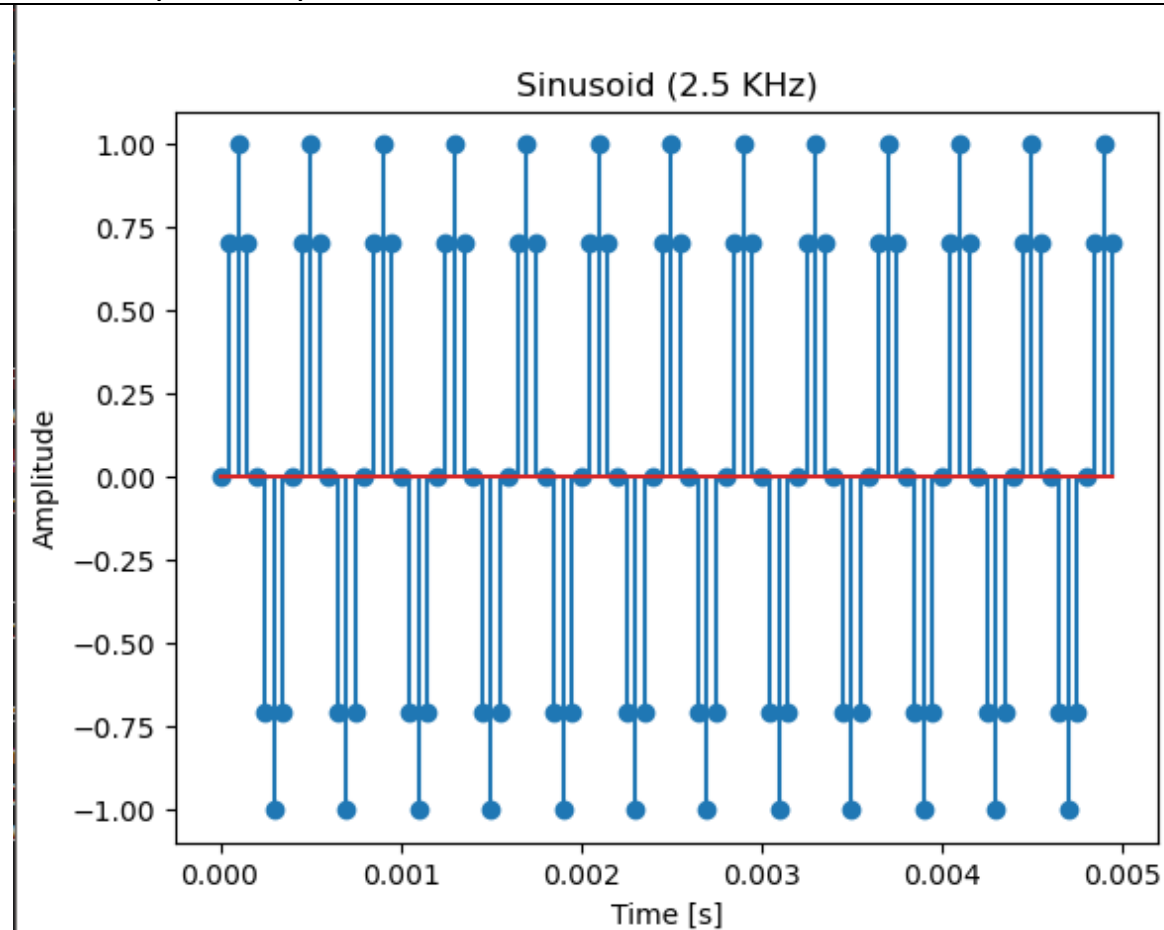
```
plot_signal(t, x, 'Random Signal with Gaussian Distribution')
```

```
X = np.fft.fft(x)
```

```
frequencies = np.fft.fftfreq(len(x), 1/fs)
```

```
plot_spectrum(frequencies, X, 'Random Signal')
```

The results (Screenshot)



Conclusion

In conclusion, the discrete Fourier series transform (DFT) is a valuable mathematical tool used to analyze and represent discrete-time signals or sequences in terms of their frequency components. It is a discrete counterpart of the continuous Fourier series transform, tailored for discrete-time systems.